



Student Dropout Detection

Apresentação Final do Projeto

IAA - Universidade de Aveiro

- Henrique Reveles 119786
- Gonçalo Simões 119412
- Giovanni Santos 115637

Este notebook analisa o **abandono escolar** e o **sucesso académico** dos estudantes.

Como apresentação final do projeto semestral, o trabalho foi concluído com uma análise abrangente de **Machine Learning** que apoia a compreensão dos fatores associados ao dropout e a tomada de decisão.

Configuração Inicial

Esta célula importa todas as bibliotecas usadas ao longo do notebook, — desde a manipulação de dados até ao modelado e à visualização — define um estado aleatório fixo para garantir reprodutibilidade e ajusta as opções de visualização.

```
# Célula 1 - Configuracao inicial do ambiente
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Evita avisos repetidos que podem dificultar a leitura
warnings.filterwarnings('ignore')

# Valor fixo para garantir que a divisao treino/teste e os modelos sao reproduziveis.
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Mostra mais colunas quando fazemos display de tabelas no notebook.
pd.set_option('display.max_columns', 80)
sns.set_theme(style='whitegrid')

print('Configuração concluída.')
```

✓ 0.0s

Importação de bibliotecas

Carrega as ferramentas necessárias para tratamento de dados, modelação e visualização.

Reprodutibilidade

Define um random state fixo para que os resultados possam ser replicados.

Opções de visualização

Ajusta parâmetros gráficos para melhorar a leitura e consistência das figuras.

Carregamento do Dataset

Esta célula procura automaticamente o ficheiro CSV na pasta **data** e carrega-o para um **DataFrame**. A lógica foi escrita para funcionar mesmo quando o notebook é aberto a partir de diretórios diferentes, tornando o projeto mais flexível.

```
PROJECT_ROOT = Path.cwd()
DATA_DIR = PROJECT_ROOT / 'data'

# Permite executar o notebook tanto na pasta iaa-project-1 como na pasta acima.
if not DATA_DIR.exists() and (PROJECT_ROOT / 'iaa-project-1' / 'data').exists():
    PROJECT_ROOT = PROJECT_ROOT / 'iaa-project-1'
    DATA_DIR = PROJECT_ROOT / 'data'

def find_first_csv(data_dir: Path):
    if not data_dir.exists():
        return None
    csv_files = sorted(data_dir.glob('*.csv'))
    return csv_files[0] if csv_files else None

DATA_PATH = find_first_csv(DATA_DIR)
TARGET_COL = 'Target'

if DATA_PATH is None:
    print("Não foi encontrado nenhum ficheiro CSV em 'data/'.")
    print('Coloca o dataset na pasta data/ ou define DATA_PATH manualmente.')
else:
    print(f'Dataset encontrado: {DATA_PATH}')

df = pd.read_csv(DATA_PATH, encoding='utf-8-sig') if DATA_PATH is not None else pd.DataFrame()

print('Dimensão do dataset:', df.shape)
```

O ponto-chave é garantir que os dados entram corretamente no fluxo de análise, servindo de base segura para os passos seguintes.

Primeira Inspeção dos Dados

Esta célula mostra as primeiras linhas do dataset, os tipos de dados, os valores em falta e as linhas duplicadas. Trata-se da etapa básica de validação da qualidade dos dados.

Antes de avançar para a modelação, é essencial confirmar se o dataset está limpo e coerente, identificando desde cedo problemas que possam comprometer a análise.

Primeiras Linhas

Permitem observar rapidamente a estrutura inicial do dataset e confirmar se as colunas estão a ser importadas corretamente.

Tipos de Dados

A verificação dos tipos de dados ajuda a identificar inconsistências nas variáveis numéricas, categóricas ou temporais.

Valores em Falta e Duplicados

Esta inspeção revela lacunas e registos repetidos que devem ser tratados antes da modelação.

```
if df.empty:
    print('DataFrame vazio. Carrega primeiro o dataset.')
else:
    display(df.head())

    print()
    print('Tipos de dados:')
    display(df.dtypes.value_counts())

    missing_values = df.isna().sum()
    duplicate_count = df.duplicated().sum()

    if missing_values.any() or duplicate_count != 0:
        print('Erro: existem valores em falta e/ou linhas duplicadas.')
        display(missing_values[missing_values > 0].sort_values(ascending=False).head(10))
        print('Número de linhas duplicadas:', duplicate_count)
    else:
        print('Validação passou: Sem valores em falta e sem linhas duplicadas.')
```

	Marital status	Application mode	Application order	Course	Daytime/evening attendance	Previous qualification	Nac
0	1	8	5	2	1	1	
1	1	6	1	11	1	1	
2	1	1	5	5	1	1	
3	1	8	2	15	1	1	
4	2	12	1	3	0	1	

Tipos de dados:

```
int64      29
float64     5
object      1
Name: count, dtype: int64
```

Validação passou: Sem valores em falta e sem linhas duplicadas.

Estatísticas Descritivas

Esta célula calcula estatísticas das variáveis numéricas, incluindo **média**, **desvio padrão**, **mínimo** e **máximo**. Estes indicadores ajudam a visualizar a escala das variáveis e a identificar valores anómalos ou fora do padrão.

O que as estatísticas revelam

- Escala e dispersão das variáveis numéricas
- Possíveis valores anómalos ou extremos
- Base para interpretar a distribuição dos dados

Mensagem principal

Esta célula ajuda a compreender melhor a distribuição dos dados antes de avançar para gráficos e modelos. É uma etapa essencial para ganhar contexto sobre o dataset e preparar a análise exploratória.

	count	mean	std	min	25%	50%	75%	max
Marital status	4424.0	1.178571	0.605747	1.00	1.00	1.000000	1.000000	6.000000
Application mode	4424.0	6.886980	5.298964	1.00	1.00	8.000000	12.000000	18.000000
Application order	4424.0	1.727848	1.313793	0.00	1.00	1.000000	2.000000	9.000000
Course	4424.0	9.899186	4.331792	1.00	6.00	10.000000	13.000000	17.000000
Daytime/evening attendance	4424.0	0.890823	0.311897	0.00	1.00	1.000000	1.000000	1.000000
Previous qualification	4424.0	2.531420	3.963707	1.00	1.00	1.000000	1.000000	17.000000
Nationality	4424.0	1.254521	1.748447	1.00	1.00	1.000000	1.000000	21.000000
Mother's qualification	4424.0	12.322107	9.026251	1.00	2.00	13.000000	22.000000	29.000000
Father's qualification	4424.0	16.455244	11.044800	1.00	3.00	14.000000	27.000000	34.000000

Distribuição da Variável Alvo

Esta célula analisa a coluna **Target**, que representa o desfecho acadêmico dos estudantes: **Dropout**, **Enrolled** ou **Graduate**. Aqui são observados o **número de ocorrências** e a **percentagem** de cada classe, além da geração de um **bar chart** para visualizar a distribuição.

O que a distribuição revela

- Quantidade de estudantes em cada classe
- Percentual relativo de Dropout, Enrolled e Graduate
- Possível desbalanceamento entre classes

Por que isso importa

O objetivo é identificar se existe **class imbalance**, já que isso influencia a escolha das **métricas de avaliação** e a forma como o desempenho do modelo deve ser interpretado.

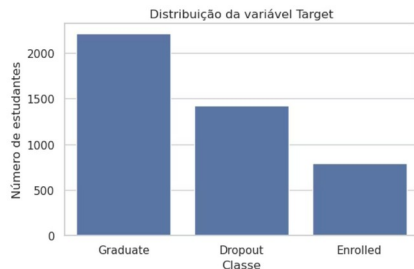
```
# Célula 5 - Análise da distribuição da variável alvo
TARGET_COL = 'Target'

if df.empty:
    print('DataFrame vazio.')
elif TARGET_COL not in df.columns:
    print(f'Coluna "{TARGET_COL}" não encontrada.')
    print('Colunas disponíveis:')
    display(df.columns.to_series())
else:
    target_counts = df[TARGET_COL].value_counts(dropna=False)
    target_percentages = df[TARGET_COL].value_counts(normalize=True, dropna=False).mul(100).round(2)

    target_summary = pd.DataFrame({
        'contagem': target_counts,
        'percentagem': target_percentages
    })
    display(target_summary)

    plt.figure(figsize=(6, 4))
    sns.countplot(data=df, x=TARGET_COL, order=target_counts.index)
    plt.title('Distribuição da variável Target')
    plt.xlabel('Classe')
    plt.ylabel('Número de estudantes')
    plt.tight_layout()
    plt.show()
```

	contagem	percentagem
Target		
Graduate	2209	49.93
Dropout	1421	32.12
Enrolled	794	17.95



Relações entre Variáveis Acadêmicas e Target

Esta célula observa variáveis acadêmicas importantes, como as **curricular units approved** e as **grades** dos dois semestres. O notebook cria **boxplots** para comparar estes indicadores entre as classes do **Target**.

A ideia principal é perceber se o desempenho acadêmico muda de forma visível entre estudantes que fazem **dropout**, permanecem **enrolled** ou fazem **graduate**.

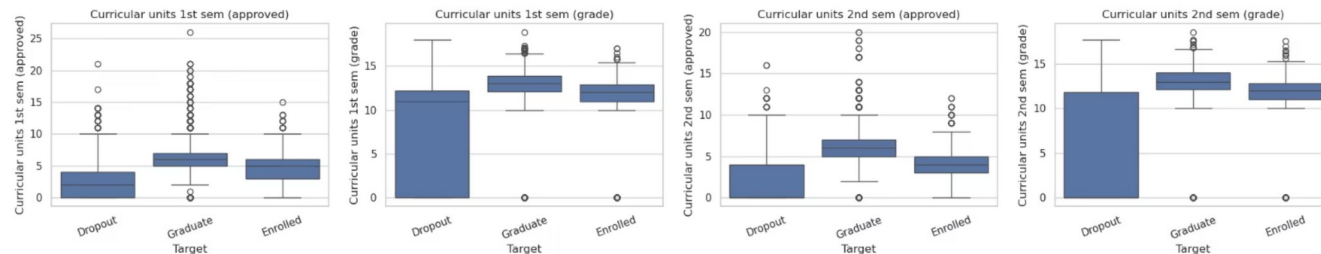
```
# Celula 6 - Exploracao visual de variaveis academicas relevantes
academic_features = [
    'Curricular units 1st sem (approved)',
    'Curricular units 1st sem (grade)',
    'Curricular units 2nd sem (approved)',
    'Curricular units 2nd sem (grade)'
]

if df.empty or TARGET_COL not in df.columns:
    print('Dados insuficientes para gerar graficos.')
else:
    available_features = [col for col in academic_features if col in df.columns]

    if not available_features:
        print('Nenhuma das variaveis academicas esperadas foi encontrada.')
    else:
        fig, axes = plt.subplots(1, len(available_features), figsize=(5 * len(available_features), 4))
        if len(available_features) == 1:
            axes = [axes]

        for ax, feature in zip(axes, available_features):
            sns.boxplot(data=df, x=TARGET_COL, y=feature, ax=ax)
            ax.set_title(feature)
            ax.set_xlabel('Target')
            ax.tick_params(axis='x', rotation=20)

        plt.tight_layout()
        plt.show()
```



Matriz de Correlação

Esta célula calcula a **correlação** entre as variáveis numéricas selecionadas e apresenta os resultados numa **heatmap**. O objetivo é identificar relações fortes entre atributos, perceber quais se movem em conjunto e detetar possíveis redundâncias entre variáveis.

Relações Fortes

Destaca pares de variáveis com correlação elevada, indicando que tendem a variar de forma semelhante.

Movimento em Conjunto

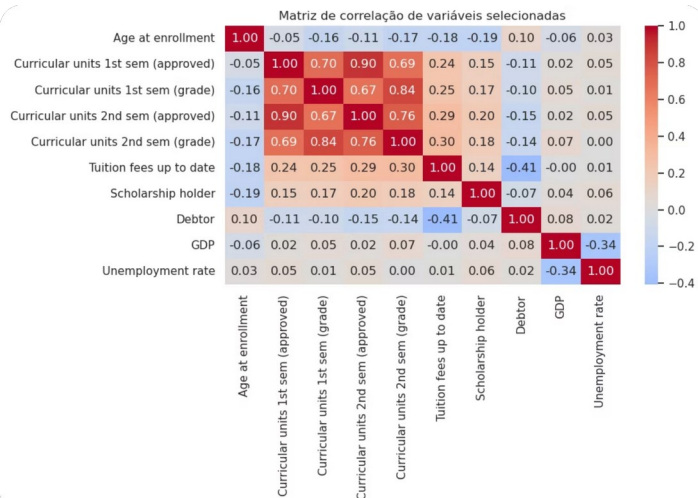
Ajuda a entender quais atributos evoluem lado a lado, revelando padrões úteis para exploração dos dados.

Redundâncias

Permite notar variáveis potencialmente redundantes, sugerindo colinearidade ou informação sobreposta.

```
# Célula 7 - Correlação entre variáveis numéricas selecionadas
if df.empty:
    print('DataFrame vazio.')
else:
    numeric_df = df.select_dtypes(include=np.number)
    selected_numeric_cols = [
        'Age at enrollment',
        'Curricular units 1st sem (approved)',
        'Curricular units 1st sem (grade)',
        'Curricular units 2nd sem (approved)',
        'Curricular units 2nd sem (grade)',
        'Tuition fees up to date',
        'Scholarship holder',
        'Debtor',
        'GDP',
        'Unemployment rate'
    ]
    selected_numeric_cols = [col for col in selected_numeric_cols if col in numeric_df.columns]

    if len(selected_numeric_cols) < 2:
        print('Variáveis numéricas insuficientes para calcular correlação.')
    else:
        corr = numeric_df[selected_numeric_cols].corr()
        plt.figure(figsize=(10, 7))
        sns.heatmap(corr, cmap='coolwarm', center=0, annot=True, fmt='.2f')
        plt.title('Matriz de correlação de variáveis selecionadas')
        plt.tight_layout()
        plt.show()
```



Importante: Correlação ajuda na exploração, mas *não implica causalidade*.

Preparação do Problema Binário

Esta célula simplifica o problema para uma **classificação binária**. A nova variável `target_binary` recebe **1** para *Dropout* e **0** para os restantes casos.

Transformação do Alvo

O objetivo original é convertido numa formulação mais direta: prever se o estudante vai **abandonar** ou **não abandonar** o curso. Esta redefinição facilita a modelação e a interpretação dos resultados.

Divisão em Treino e Teste

O dataset é dividido em conjuntos de treino e teste com **stratify**, preservando a proporção das classes em ambos os subconjuntos. Isso ajuda a manter uma avaliação mais equilibrada e representativa.

O ponto central desta célula é transformar o objetivo original num problema binário mais claro: **prever dropout ou não-dropout**.

```
# Celula 8 - Criacao do alvo binario e divisao treino/teste
if df.empty or TARGET_COL not in df.columns:
    print('Nao e possivel preparar dados sem dataset ou coluna Target.')
else:
    work_df = df.copy()

    # 1 significa estudante em abandono; 0 significa estudante inscrito ou graduado.
    work_df['target_binary'] = np.where(work_df[TARGET_COL].astype(str).str.lower() == 'dropout', 1, 0)

    X = work_df.drop(columns=[TARGET_COL, 'target_binary'])
    y_binary = work_df['target_binary']

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y_binary,
        test_size=0.2,
        random_state=RANDOM_STATE,
        stratify=y_binary
    )

    print('Tamanho do treino:', X_train.shape)
    print('Tamanho do teste:', X_test.shape)
    print()
    print('Distribuicao do alvo binario no treino:')
    display(y_train.value_counts(normalize=True).rename('proporcao'))
```

✓ 0.0s

Tamanho do treino: (3539, 34)

Tamanho do teste: (885, 34)

Distribuicao do alvo binario no treino:

target_binary

0 0.678723

1 0.321277

Name: proporcao, dtype: float64

Pipeline de Pré-processamento

Esta célula define o pré-processamento utilizado antes dos modelos. Variáveis numéricas recebem imputação por mediana e normalização, enquanto variáveis categóricas recebem imputação por valor mais frequente e one-hot encoding. A mensagem principal é que este pipeline organiza o tratamento dos dados de forma robusta e automática.

Variáveis Numéricas

Imputação por mediana e normalização para lidar com valores em falta e padronizar a escala.

Variáveis Categóricas

Imputação por valor mais frequente e one-hot encoding para converter categorias em formato numérico.

```
# Célula 9 - Definição do pré-processamento
if df.empty or TARGET_COL not in df.columns:
    print('Dados insuficientes para criar o pré-processamento.')
else:
    numeric_selector = make_column_selector(dtype_include=np.number)
    categorical_selector = make_column_selector(dtype_exclude=np.number)

    numeric_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    categorical_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])

    preprocessor = ColumnTransformer([
        ('numeric', numeric_pipeline, numeric_selector),
        ('categorical', categorical_pipeline, categorical_selector)
    ])

    print('Pré-processamento definido.')
```

0.0s

Comparação e Avaliação de Modelos

Aqui são comparados os modelos testados: **Regressão Logística**, **Random Forest**, **AdaBoost**, **Gradient Boosting**, **HistGradient Boosting** e **Dummy Baseline**. A avaliação usou validação cruzada estratificada e múltiplas métricas para garantir uma comparação consistente entre os candidatos.

Modelos Testados

Regressão Logística, Random Forest, AdaBoost, Gradient Boosting, HistGradient Boosting e Dummy Baseline.

Vencedor

Gradient Boosting foi selecionado como modelo final, com **F1-score médio em CV de 79,09%**.

Random Forest

Mostrou overfitting: **100% de F1 no treino** versus **77.41% de F1 em CV**.

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=2000, class_weight='balanced', random_state=RANDOM_STATE),
    'Random Forest': RandomForestClassifier(n_estimators=250, class_weight='balanced', random_state=RANDOM_STATE),
    'AdaBoost': AdaBoostClassifier(n_estimators=150, learning_rate=0.05, random_state=RANDOM_STATE),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=150, learning_rate=0.05, max_depth=3, random_state=RANDOM_STATE),
    'Hist Gradient Boosting': HistGradientBoostingClassifier(max_iter=150, learning_rate=0.05, class_weight='balanced', random_state=RANDOM_STATE)
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = ['accuracy', 'balanced_accuracy', 'f1', 'precision', 'recall', 'roc_auc']
cv_results = {}

for model_name, model in models.items():
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=1
    )

    summary = {metric: float(np.mean(scores[f'test_{metric}'])) for metric in scoring}
    summary['model'] = model_name
    cv_results.append(summary)

cv_results_df = pd.DataFrame(cv_results).set_index('model').sort_values('f1', ascending=False)
cv_results_percent = (cv_results_df * 100).round(2)

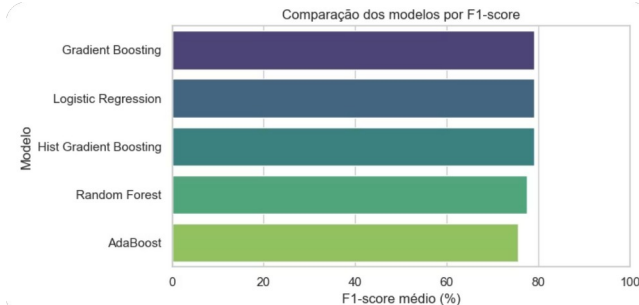
print('Comparação por validação cruzada estratificada, valores em percentagem:')
display(
    cv_results_percent.style
        .format('{:.2f}%')
        .highlight_max(axis=0, color='#d7f0d2')
        .set_caption('Resultados médios em 5 folds')
)

best_model_name = cv_results_df.index[0]
print(f'Melhor modelo por F1-score: {best_model_name} ((cv_results_percent.loc[best_model_name, "f1"]:.2f)%)')
```

Comparação por validação cruzada estratificada, valores em percentagem:

model	Resultados médios em 5 folds					
	accuracy	balanced_accuracy	f1	precision	recall	roc_auc
Gradient Boosting	87.62%	83.73%	79.09%	86.57%	72.82%	91.79%
Logistic Regression	86.13%	84.89%	79.08%	76.98%	81.44%	91.87%
Hist Gradient Boosting	86.63%	84.41%	78.99%	79.82%	78.19%	91.46%
Random Forest	86.95%	82.39%	77.41%	87.16%	69.65%	91.53%
AdaBoost	85.28%	81.44%	75.51%	81.03%	70.71%	89.91%

Melhor modelo por F1-score: Gradient Boosting (79.09%)



O **Gradient Boosting** foi escolhido por oferecer o melhor equilíbrio entre desempenho e generalização, enquanto o Random Forest perdeu robustez fora do treino.

Avaliação no Conjunto de Teste

Depois de selecionar o **Gradient Boosting** como melhor modelo com base na validação cruzada, o notebook o treina no conjunto de treino completo e o avalia no conjunto de teste. Esta célula mostra o relatório de classificação e a matriz de confusão com os resultados finais.

01

Seleção do Gradient Boosting

O modelo foi escolhido como o melhor candidato após a validação cruzada.

03

Avaliação no Conjunto de Teste

No teste, o modelo alcança **87,57% de accuracy**, mostrando forte capacidade de generalização.

Os resultados no conjunto de teste confirmam o **bom desempenho do Gradient Boosting**, com accuracy elevada e métricas sólidas para a classe Dropout, indicando um modelo consistente e confiável para dados novos.

```
if df.empty or 'TARGET' not in df.columns:
    print('Dados insuficientes para a avaliação final.')
else:
    best_model_name = cv_results_df.index[0]
    best_model = model_dict[best_model_name]

    best_pipeline = Pipeline([
        (preprocessor, preprocessor),
        (model, best_model)
    ])

    best_pipeline.fit(X_train, y_train)
    y_pred = best_pipeline.predict(X_test)

    report = classification_report(
        y_test,
        y_pred,
        target_names=['Non-Dropout', 'Dropout'],
        output_dict=True,
        zero_division=0
    )

    report_df = pd.DataFrame(report).T
    metrics_cols = ['precision', 'recall', 'f1_score']
    report_row = ['Non-Dropout', 'Dropout', 'macro avg', 'weighted avg']
    report_percent = report_df.loc[report_row, metrics_cols].T.copy()
    report_percent[metrics_cols] = report_percent[metrics_cols] * 100
    report_percent['support'] = report_percent['support'].astype(int)
    test_accuracy = report['accuracy'] * 100

    print('Melhor modelo:', best_model_name)
    print(f'Accuracy no conjunto de teste (test accuracy: 27%)')
    print('Resultados no conjunto de teste:')
    display(report_percent.style
             .format(precision='{:.2f}%', recall='{:.2f}%', f1_score='{:.2f}%', support='{:.0f}')
             .highlight_null(isnull=metrics_cols, axis=0, color='red'))

    cm = confusion_matrix(y_test, y_pred)
    cm_percent = confusion_matrix(y_test, y_pred, normalize='true') * 100
    cm_labels = np.array([
        ['Non-Dropout', 'Dropout'] for i in range(cm.shape[1])
    ])

    plt.figure(figsize=(5, 4))
    cm_percent.heatmap(
        ax=cm_percent,
        annot=True,
        cbar=cm_percent,
        cmap='Blues',
        vmin=0,
        cbar_kw=dict(label='Porcentagem por classe real'),
        xticklabels=['Non-Dropout', 'Dropout'],
        yticklabels=['Non-Dropout', 'Dropout']
    )

    plt.title('Matriz de confusão (%) - problema binário')
    plt.xlabel('Previsto')
    plt.ylabel('Real')
    plt.tight_layout()
    plt.show()
```

02

Treino com Todo o Conjunto de Treino

O **Gradient Boosting** é treinado novamente usando todos os dados de treino disponíveis.

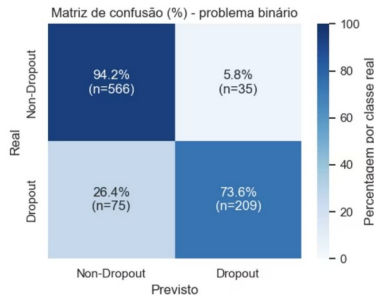
04

Relatório e Matriz de Confusão

O relatório destaca a classe **Dropout** com **F1-score de 79,55%** e **Recall de 75,35%**.

Melhor modelo: Gradient Boosting
Accuracy no conjunto de teste: 87.57%
Resultados no conjunto de teste:

	precision	recall	f1-score	support
Non-Dropout	88.30%	94.18%	91.14%	601
Dropout	85.66%	73.59%	79.17%	284
macro avg	86.98%	83.88%	85.15%	885
weighted avg	87.45%	87.57%	87.30%	885



Importância das Variáveis

Esta célula extrai e visualiza as variáveis mais importantes para o modelo **Gradient Boosting**. A análise foi feita com **Permutation Importance**, uma técnica que ajuda a perceber como o modelo toma decisões e quais fatores têm maior impacto nas previsões de abandono.



Permutation Importance no Gradient Boosting

Identifica, de forma interpretável, as variáveis que mais influenciam as previsões do modelo.

```
permutation = permutation_importance(
    best_pipeline,
    X_test,
    y_test,
    n_repeats=3,
    random_state=RANDOM_STATE,
    scoring='f1',
    n_jobs=1
)
importances = pd.Series(permutation.importances_mean, index=X_test.columns)
top_importances = importances.sort_values(ascending=False).head(15)

print(f'Importância das variáveis calculada para: {best_model.name}')
display(top_importances.rename('importance').to_frame().style.format(':.4f'))

plt.figure(figsize=(8, 6))
sns.barplot(x=top_importances.values, y=top_importances.index)
```

Importância das variáveis calculada para: Gradient Boosting

	importance
Curricular units 2nd sem (approved)	0.2320
Tuition fees up to date	0.0770
Scholarship holder	0.0116
Age at enrollment	0.0075
Curricular units 1st sem (approved)	0.0070
Unemployment rate	0.0050
Curricular units 2nd sem (credited)	0.0045
Curricular units 1st sem (credited)	0.0043
GDP	0.0030
Curricular units 1st sem (evaluations)	0.0018
Mother's qualification	0.0018
Curricular units 1st sem (grade)	0.0003
International	0.0000
Inflation rate	0.0000
Curricular units 2nd sem (without evaluations)	0.0000



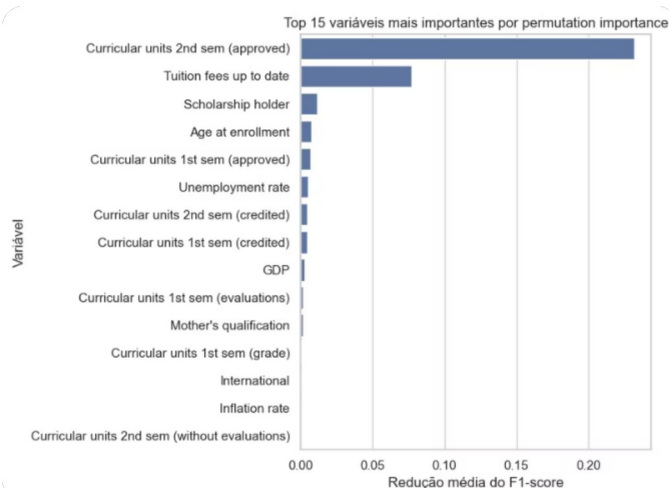
Top 5 Variáveis Mais Importantes

Curricular units 2nd sem (approved), Tuition fees up to date, Scholarship holder, Age at enrollment e Curricular units 1st sem (approved).



Principais Fatores das Previsões

Essas são as variáveis-chave que mais explicam as decisões do **Gradient Boosting** sobre o abandono.



Baseline e Controlo de Overfitting

Esta análise focou-se em garantir que o modelo selecionado não estivesse a "decorar" os dados de treino, mas sim a aprender padrões generalizáveis. A validação cruzada foi crucial para monitorizar e comparar o desempenho dos modelos.

Deteção de Overfitting

Comparámos cuidadosamente os F1-scores obtidos no conjunto de treino com os resultados da validação cruzada para cada modelo.

Desempenho do Gradient Boosting

O Gradient Boosting não apresentou sinais significativos de overfitting ou underfitting, indicando que aprendeu padrões reais nos dados de forma consistente.

O Contraste com Random Forest

O modelo Random Forest revelou memorização do conjunto de treino (100% de F1 no treino vs. 77.41% em CV), um claro sinal de overfitting.

Esta distinção é fundamental: enquanto um modelo memorizador falha em novos dados, o Gradient Boosting demonstra robustez e capacidade de generalização para prever o abandono escolar.

```
diagnostic_models = {
    'Dummy baseline': DummyClassifier(strategy='most_frequent'),
    **models
}

diagnostic_rows = []
for model_name, model in diagnostic_models.items():
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring=({'f1': 'f1', 'balanced_accuracy': 'balanced_accuracy'},
        return_train_score=True,
        n_jobs=1
    )

    diagnostic_rows.append([
        'model': model_name,
        'train_f1': scores['train_f1'].mean(),
        'cv_f1': scores['test_f1'].mean(),
        'gap_f1': scores['train_f1'].mean() - scores['test_f1'].mean(),
        'cv_balanced_accuracy': scores['test_balanced_accuracy'].mean()
    ])

overfitting_df = pd.DataFrame(diagnostic_rows).set_index('model').sort_values('cv_f1', ascending=False)
overfitting_percent = (overfitting_df * 100).round(2)

print("Diagnóstico treino vs validação cruzada, valores em percentagem:")
display(
    overfitting_percent.style
        .format("{:.2f}%")
        .highlight_max(subset=['cv_f1', 'cv_balanced_accuracy'], color="#d7f0d2")
        .highlight_min(subset=['gap_f1'], color="#d7f0d2")
)
```

Diagnóstico treino vs validação cruzada, valores em percentagem:

	train_f1	cv_f1	gap_f1	cv_balanced_accuracy
model				
Gradient Boosting	82.64%	79.09%	3.55%	83.73%
Logistic Regression	80.02%	79.08%	0.94%	84.89%
Hist Gradient Boosting	96.32%	78.99%	17.33%	84.41%
Random Forest	100.00%	77.41%	22.59%	82.39%
AdaBoost	75.52%	75.51%	0.01%	81.44%
Dummy baseline	0.00%	0.00%	0.00%	50.00%

Interpretação para Gradient Boosting: sem sinal forte de overfitting ou underfitting nos resultados médios.

Avaliação Final do Melhor Modelo

Após a seleção criteriosa, o modelo **Gradient Boosting** passa pela sua avaliação final no conjunto de teste, garantindo uma estimativa de desempenho totalmente imparcial.

Estimativa de Desempenho Honesta

A separação rigorosa do conjunto de teste da fase de treino e validação cruzada foi fundamental para obter uma estimativa de desempenho **imparcial**, refletindo com precisão a performance do modelo em dados desconhecidos.

Métricas Finais Robustas

O **Gradient Boosting** validou sua eficácia, alcançando uma **accuracy de 87,57%** no conjunto de teste. Sua capacidade de identificar a classe **Dropout** permanece forte, fornecendo previsões valiosas.

A avaliação final confirma a **confiabilidade e a robustez** do modelo, solidificando a credibilidade dos resultados para prever e mitigar o abandono escolar.

```
final_pipeline = best_pipeline
final_model_name = best_model_name
final_y_pred = final_pipeline.predict(X_test)
final_report = classification_report(
    y_test,
    final_y_pred,
    target_names=['Non-Dropout', 'Dropout'],
    output_dict=True,
    zero_division=0
)
final_report_df = pd.DataFrame(final_report).T
final_report_percent = final_report_df.loc[report_rows, metric_cols + ['support']].copy()
final_report_percent[metric_cols] = final_report_percent[metric_cols] * 100
final_report_percent['support'] = final_report_percent['support'].astype(int)

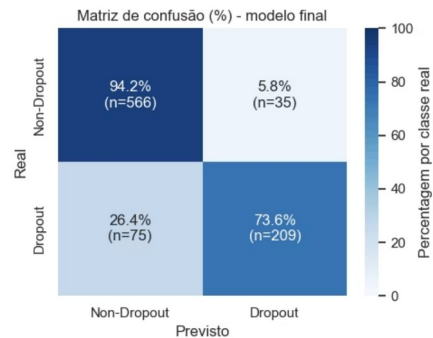
print('Modelo final:', final_model_name)
print(f'Accuracy no teste: {final_report["accuracy"] * 100:.2f}%')
display(
    final_report_percent.style
    .format({'precision': '{:.2f}%', 'recall': '{:.2f}%', 'f1-score': '{:.2f}%', 'support': '{:.0f}'})
    .highlight_max(subset=metric_cols, axis=0, color="#d7f0d2")
)

final_cm = confusion_matrix(y_test, final_y_pred)
final_cm_percent = confusion_matrix(y_test, final_y_pred, normalize='true') * 100
final_cm_labels = np.array([
    [f'{final_cm_percent[i, j]:.1f}%\n(n={final_cm[i, j]})' for j in range(final_cm.shape[1])]
    for i in range(final_cm.shape[0])
])

plt.figure(figsize=(5, 4))
sns.heatmap(
    final_cm_percent,
    annot=final_cm_labels,
    fmat='',
    cmap='Blues',
    vmin=0,
    vmax=100,
    cbar_hws=('Percentagem por classe real',),
    xticklabels=['Non-Dropout', 'Dropout'],
    yticklabels=['Non-Dropout', 'Dropout']
)
plt.title('Matriz de confusão (%) - modelo final')
plt.xlabel('Previsto')
plt.ylabel('Real')
plt.tight_layout()
```

Modelo final: Gradient Boosting
Accuracy no teste: 87.57%

	precision	recall	f1-score	support
Non-Dropout	88.30%	94.18%	91.14%	601
Dropout	85.66%	73.59%	79.17%	284
macro avg	86.98%	83.88%	85.15%	885
weighted avg	87.45%	87.57%	87.30%	885



A avaliação estendeu-se aos resultados originais do dataset: **Desistente**, **Matriculado** e **Graduado**.



O **Hist Gradient Boosting** destacou-se com um **F1-macro de 71,74%**, provando sua eficácia no cenário multiclasse.



Este resultado valida a adaptabilidade das nossas escolhas de modelagem para diferentes formulações do problema.

```
x_multi = df.drop(columns=[TARGET_COL])
y_multi = df[TARGET_COL]

X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(
    X_multi,
    y_multi,
    test_size=0.3,
    random_state=RANDOM_STATE,
    stratify=y_multi
)

multiclass_models = [
    'Logistic Regression: LogisticRegression(max_iter=1000, class_weight='balanced', random_state=RANDOM_STATE)',
    'Random Forest: RandomForestClassifier(n_estimators=250, class_weight='balanced', random_state=RANDOM_STATE)',
    'XGBoost: XGBClassifier(n_estimators=100, class_weight='balanced', random_state=RANDOM_STATE)',
    'AdaBoost: AdaBoostClassifier(n_estimators=100, learning_rate=0.5, class_weight='balanced')
]

multiclass_rows = []
for model_name, model in multiclass_models.items():
    pipeline = Pipeline([
        (preprocessor, preprocessor),
        (model, model)
    ])
    scores = cross_validate(
        pipeline,
        X_train_multi,
        y_train_multi,
        cv=cv,
        scoring=['accuracy', 'balanced_accuracy', 'f1_macro'],
        n_jobs=1
    )
    multiclass_rows.append({
        'model': model_name,
        'accuracy': scores['test_accuracy'].mean(),
        'balanced_accuracy': scores['test_balanced_accuracy'].mean(),
        'f1_macro': scores['test_f1_macro'].mean()
    })

multiclass_results_df = pd.DataFrame(multiclass_rows).set_index('model').sort_values('f1_macro', ascending=False)
multiclass_percent = multiclass_results_df['f1_macro'].round(2)

print('Comparação multiclasse por validação cruzada, valores em porcentagem:')
display(
    multiclass_percent.style
    .format('{:.2f}%')
    .highlight_max(axis=0, color='#00f002')
)

)

best_multiclass_name = multiclass_results_df.index[0]
best_multiclass_pipeline = Pipeline([
    (preprocessor, preprocessor),
    (model, multiclass_models[best_multiclass_name])
])

best_multiclass_pipeline.fit(X_train_multi, y_train_multi)
y_pred_multi = best_multiclass_pipeline.predict(X_test_multi)

multi_report = classification_report(y_test_multi, y_pred_multi, output_dict=True, zero_division=0)
multi_report = pd.DataFrame(multi_report)

multi_classes = best_multiclass_pipeline.named_steps['model'].classes
multi_rows = [list(multi_classes) + ['macro avg', 'weighted avg']]
multi_report['multi_report'] = multi_report['f1_macro'].round(2).str.replace(' ', 'support')
multi_report['multi_report'] = multi_report['multi_report'].str.replace(' ', 'support')
multi_report['multi_report'] = multi_report['multi_report'].str.replace(' ', 'support')

print('Melhor modelo multiclasse: ', best_multiclass_name)
display(
    multi_report.percent.format(
        'precision: {:.2f}%', 'recall: {:.2f}%', 'f1-score: {:.2f}%', 'support: {:.0f}')
    )
)
```

	accuracy	balanced_accuracy	f1_macro
model			
Hist Gradient Boosting	77.08%	71.35%	71.74%
Logistic Regression	74.82%	71.32%	70.62%
Random Forest	77.56%	68.08%	69.24%

	precision	recall	f1-score	support
Dropout	81.75%	72.54%	76.87%	284
Enrolled	46.81%	55.35%	50.72%	159
Graduate	83.60%	84.16%	83.88%	442
macro avg	70.72%	70.68%	70.49%	885
weighted avg	76.39%	75.25%	75.67%	885

Desempenho por Subgrupos e Responsabilidade

Para assegurar a equidade e mitigar potenciais vieses, avaliamos o desempenho do modelo em diversos subgrupos, focando na responsabilidade e ética da IA.

Teste de Equidade em Subgrupos

A justiça do modelo foi analisada nos subgrupos: **Gênero, Bolsista, Devedor e Propinas em dia.**

Apoio à Intervenção Humana

O modelo deve servir **apenas como apoio** à decisão e intervenção humana, não substituí-la.

Lacunas e IA Responsável

Foram identificadas **lacunas no recall** entre os grupos, realçando a necessidade de uma aplicação responsável da IA.

Evitar Decisões Automatizadas

É fundamental que o modelo **não seja usado para decisões automatizadas**, mantendo a supervisão humana.

Esta análise reforça o compromisso com a **IA ética**, garantindo que a tecnologia atue como uma ferramenta de apoio, com a **supervisão humana primordial** para mitigar vieses e promover a justiça e a equidade nos resultados.

```
subgroup_columns = ['Gender', 'Scholarship holder', 'Debtor', 'Tuition fees up to date']
subgroup_columns = [col for col in subgroup_columns if col in X_test.columns]

subgroup_rows = []
for col in subgroup_columns:
    for value in sorted(X_test[col].dropna().unique()):
        mask = X_test[col] == value
        support = int(mask.sum())
        positives = int(y_test[mask].sum())
        if support < 30 or len(np.unique(y_test[mask])) < 2:
            continue
        subgroup_rows.append({
            'subgroup': col,
            'value': value,
            'support': support,
            'dropout_rate': y_test[mask].mean(),
            'recall_dropout': classification_report(
                y_test[mask],
                final_y_pred[mask],
                output_dict=True,
                zero_division=0
            )['1']['recall'], # pyright: ignore[reportArgumentType]
            'f1_dropout': f1_score(y_test[mask], final_y_pred[mask], zero_division=0)
        })

subgroup_df = pd.DataFrame(subgroup_rows)
if subgroup_df.empty:
    print('Não há subgrupos com dimensão suficiente para uma leitura estável.')
else:
    subgroup_display = subgroup_df.copy()
    for col in ['dropout_rate', 'recall_dropout', 'f1_dropout']:
        subgroup_display[col] = subgroup_display[col] * 100

    display(
        subgroup_display.sort_values(['subgroup', 'value']).style
        .format({'dropout_rate': '{:.2f}%', 'recall_dropout': '{:.2f}%', 'f1_dropout': '{:.2f}%'})
    )

    fairness_summary = subgroup_df.groupby('subgroup')['recall_dropout'].agg(['min', 'max'])
    fairness_summary['gap'] = fairness_summary['max'] - fairness_summary['min']
    fairness_summary = (fairness_summary * 100).round(2)
    print('Diferença de recall da classe Dropout entre valores de cada subgrupo:')
    display(fairness_summary.style.format('{:.2f}%'))
```

	subgroup	value	support	dropout_rate	recall_dropout	f1_dropout
4	Debtor	0	795	29.18%	70.26%	77.62%
5	Debtor	1	90	57.78%	88.46%	85.19%
0	Gender	0	564	24.47%	72.46%	78.12%
1	Gender	1	321	45.48%	74.66%	80.15%
2	Scholarship holder	0	654	38.99%	75.69%	80.58%
3	Scholarship holder	1	231	12.55%	55.17%	65.31%
6	Tuition fees up to date	0	114	85.96%	97.96%	95.05%
7	Tuition fees up to date	1	771	24.12%	60.75%	69.33%

Diferença de recall da classe Dropout entre valores de cada subgrupo:

	min	max	gap
subgroup			
Debtor	70.26%	88.46%	18.20%
Gender	72.46%	74.66%	2.19%
Scholarship holder	55.17%	75.69%	20.51%
Tuition fees up to date	60.75%	97.96%	37.21%

Conclusão

O nosso projeto seguiu uma sequência lógica rigorosa para garantir um modelo robusto e ético:

01

1. Problema

Definição clara do desafio do abandono escolar.

02

2. Dados

Coleta, pré-processamento e análise exploratória.

03

3. Pipeline

Desenvolvimento e otimização do fluxo de trabalho do modelo.

04

4. Avaliação

Métricas de desempenho e validação cruzada para garantir robustez.

05

5. Melhor Modelo

Seleção do Gradient Boosting pelo seu equilíbrio entre precisão e generalização.

06

6. Interpretação

Análise da importância das variáveis para compreensão do modelo.

07

7. Limitações e Ética

Reconhecimento de vieses e a necessidade de supervisão humana.

O modelo desenvolvido é uma ferramenta valiosa para a sinalização precoce de alunos em risco de abandono, permitindo intervenções direcionadas. No entanto, o seu uso deve ser sempre acompanhado de **supervisão humana e crítica**.

- O desenvolvimento e estruturação do código contou com o apoio de ferramentas de IA, nomeadamente Codex/OpenAI, mas todas as decisões finais e a revisão crítica foram assumidas pelos estudantes.

